AI Automation in
End-to-End Data Workflows

Auto-Tagging • Report Generation • Chatbots

A practical guide on the Databricks Lakehouse Platform

Prepared for: Prachi Kabra
Platform: Databricks (Unity Catalog, Serverless, Mosaic AI)
Companion notebook: `ai_automation_databricks.ipynb`

1. What AI Automation Means in a Data Workflow

Traditional data engineering moves and reshapes data: ingest, clean, join, aggregate, serve. AI automation adds a new class of transformation — one that reasons over unstructured or semi-structured content (text, documents, images) and produces structured, decision-ready outputs without a human in the loop for every row.

The important shift is that a large language model becomes just another operator in your pipeline. Instead of writing brittle regex or maintaining a hand-labelled training set, you call a model the same way you call SUM() or a UDF, apply it across millions of rows, and land the result in a governed table. This guide focuses on three of the highest-value patterns and how they compose into an end-to-end flow.

The three patterns covered

- **Auto-tagging —** classify, route, extract entities, and score sentiment on incoming records automatically (e.g., support tickets, product reviews, documents).
- **Report generation —** turn aggregated metrics into readable narrative summaries, executive briefs, or per-segment commentary on a schedule.
- **Chatbots —** let users ask natural-language questions and get grounded answers from your own data using retrieval-augmented generation (RAG).

> **Why on Databricks?**
> The AI lives next to the data. There is no copying rows to an external API layer and back — the model runs against your governed Delta tables, results inherit Unity Catalog lineage and permissions, and the same job that transforms data can enrich it with AI in one pipeline.

2. The Databricks AI Automation Stack

You can build every pattern in this guide from four building blocks. You rarely need all four at once — auto-tagging and reports need only the first, chatbots add the rest.

Layer	What it is	Use it for
AI Functions	Built-in SQL/Python functions (ai_query, ai_classify, ai_extract, ai_summarize, ai_gen, ai_analyze_sentiment, ai_translate, ai_mask, ai_parse_document).	Auto-tagging, extraction, summaries, batch report text — the workhorses.
Model Serving / Foundation Models	Managed endpoints for Databricks-hosted (Llama, Claude, etc.) or external models, called via ai_query.	Custom prompts, full control over model and parameters, agent hosting.
Mosaic AI Vector Search	A vector index synced from a Delta table for semantic retrieval.	The 'retrieval' half of a chatbot / RAG.
Agent Framework + Workflows	Tooling to build, evaluate, and deploy agents, plus jobs/DLT to schedule everything.	Production chatbots and orchestrating the end-to-end pipeline.

> **Compute requirement (read this first)**

> The general-purpose ai_query function and the newest task functions require serverless compute (or a serverless SQL warehouse) and Databricks Runtime 18.2+. They are not available on Pro or Classic SQL warehouses. If a function errors, checking your compute type is usually the fix.

3. Pattern 1 — Auto-Tagging

Auto-tagging is the fastest win. A single SQL statement can categorise, route, score, and extract from a whole table. The task-specific functions are the simplest entry point because you do not manage a prompt or an endpoint.

3.1 Classification and routing

Use `ai_classify` to assign each row one (or more) of a fixed label set — perfect for ticket categories, product areas, or intent routing. Databricks recommends the v2.0 interface, which also accepts label descriptions to disambiguate.

```
SELECT
  ticket_id,
  body,
  ai_classify(
    body,
    '["Billing", "Bug", "Feature Request", "Account Access"]',
    map('version', '2.0')
  ) AS category
FROM support.raw.tickets;
```

3.2 Sentiment and entity extraction

Stack functions in the same query: `ai_analyze_sentiment` returns positive / negative / mixed / neutral, and `ai_extract` pulls named fields (order numbers, product names, contact details) out of free text into a struct you can flatten.

```
SELECT
  review_id,
  ai_analyze_sentiment(text)                        AS sentiment,
  ai_extract(text, ARRAY('product', 'issue', 'order_no')) AS entities
FROM shop.raw.reviews;
```

> **Design tip**
> Write AI-enriched columns into a Silver table, not on the fly at read time. You pay the inference cost once, the results become queryable and joinable like any column, and downstream BI stays fast and cheap.

4. Pattern 2 — Automated Report Generation

Reporting is where generative (rather than classifying) functions shine. The workflow is always the same two steps: compute the numbers with ordinary SQL, then hand those numbers to a model to write the prose. Never ask the model to do arithmetic — let SQL be the source of truth and let the model narrate.

4.1 Summarising long text

`ai_summarize` condenses long fields — call notes, contracts, incident logs — with an optional word budget.

```
SELECT case_id, ai_summarize(full_transcript, 60) AS tl_dr
FROM support.gold.resolved_cases;
```

4.2 Generating a narrative from metrics

`ai_gen` takes a free-form instruction. Assemble your KPIs into a string, then ask for an executive summary. Because the prompt carries the real figures, the output is grounded in your data.

```
WITH kpis AS (
  SELECT concat(
    'Revenue: $', round(sum(amount)/1e6, 2), 'M. ',
    'Orders: ', count(*), '. ',
    'Avg order: $', round(avg(amount), 2)
  ) AS summary_line
  FROM sales.gold.orders WHERE week = current_date() - 7
)
SELECT ai_gen(
  concat('Write a 3-sentence executive summary for this weekly ',
         'sales snapshot for a non-technical VP: ', summary_line)
) AS exec_summary
FROM kpis;
```

> **Guardrail**
> Keep the AI to language, not numbers. Compute every figure in SQL and pass it in; ask the model only to explain and phrase. This keeps reports auditable and eliminates the most common failure mode — a confidently wrong statistic.

5. Pattern 3 — Chatbots (RAG)

A useful enterprise chatbot answers from your data, not from the model's training. That is retrieval-augmented generation: find the most relevant chunks of your knowledge base, then ask the model to answer using only those chunks. This grounds answers and lets you cite sources.

The RAG loop

1. Chunk and embed your source documents; sync them into a Mosaic AI Vector Search index off a Delta table.
2. At query time, embed the user's question and retrieve the top-k most similar chunks.
3. Build a prompt: the retrieved context + the question, then call `ai_query` against a foundation model.
4. Return the answer with the source chunks it was grounded on.

The generation step is a single call. `ai_query` gives you full control of model, prompt, and return type:

```
SELECT ai_query(
  'databricks-llama-4-maverick',
  concat('Answer using ONLY this context. If the answer is not ',
         'present, say you do not know.\n\nContext:\n', retrieved_chunks,
         '\n\nQuestion: ', :user_question)
) AS answer;
```

Scale note
For a few dozen documents you can skip a vector index and stuff the whole knowledge base into the prompt (as the companion notebook does, for simplicity). Past a few hundred, use Vector Search so retrieval stays fast and the prompt stays within the context window. For multi-step or tool-using assistants, graduate to the Mosaic AI Agent Framework.

6. Putting It Together — The End-to-End Workflow

The three patterns are not separate products; they are stages of one governed pipeline built on the medallion architecture. AI enrichment slots naturally into the Silver and Gold layers.

Stage	Layer	What happens	AI role
Ingest	Bronze	Raw tickets, reviews, docs land as-is (Auto Loader / streaming).	None — keep raw fidelity.
Enrich	Silver	Clean + AI-tag each row: category, sentiment, extracted entities.	Auto-tagging (Pattern 1).
Aggregate	Gold	Business-level metrics and rollups by segment / period.	None — plain SQL.
Narrate	Gold+	Generate scheduled narrative reports from the Gold metrics.	Report generation (Pattern 2).
Serve	App	Chatbot answers questions grounded on Gold + docs.	RAG chatbot (Pattern 3).

Orchestration

Wrap the Silver enrichment and Gold narration as notebook tasks in a Databricks Workflow (or Lakeflow Declarative Pipelines / DLT) on a schedule. The chatbot is deployed separately as a served endpoint or app that reads the same Gold tables. One lineage graph, one permission model, one place to monitor.

Governance & cost checklist
Unity Catalog governs every AI-enriched table and endpoint. Materialise enrichment once into Silver rather than re-inferring per query. Mask PII with ai_mask before it reaches a prompt. Batch AI calls in scheduled jobs rather than interactive re-runs. Track spend per workload via the AI Gateway / usage tables.

7. Quick Reference — AI Functions

Function	Returns	Typical use
ai_query	Anything (typed)	General-purpose model call; custom prompts; chatbots.
ai_classify	Label(s)	Categorisation, routing, tagging.
ai_extract	Struct of fields	Pull entities from free text.
ai_analyze_sentiment	Sentiment label	Feedback / review scoring.

Function	Returns	Typical use
ai_summarize	Short text	Condense long documents.
ai_gen	Generated text	Narrative reports, rewrites, drafting.
ai_translate	Translated text	Localisation.
ai_mask	Masked text	Redact PII before prompting / sharing.
ai_parse_document	Structured doc	Parse PDFs/images into text + layout.
ai_similarity	Score	Compare two texts semantically.

The companion notebook — **`ai_automation_databricks.ipynb`** — implements all three patterns end-to-end on synthetic data you can run as-is on a serverless cluster.

Function names, syntax, and compute requirements reflect Databricks documentation current as of July 2026. AI functions evolve — confirm against your workspace's docs before production use.